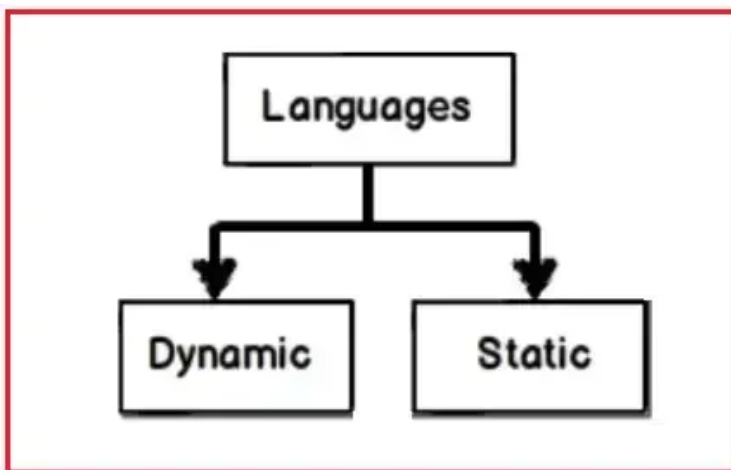


Dynamic Type in C# with Examples

In this article, I am going to discuss **Dynamic Type in C#** with Examples. Please read our previous article where we discussed [Reflection in C#](#) with Examples. As part of C# 4.0, a new type was introduced called dynamic that avoids compile-time type checking. A dynamic type escapes type checking at compile-time; instead, it resolves the type at run time.

Types of Programming Languages:

Before understanding the keyword dynamic in C#, let us first understand the different types of programming languages that exist in the market. Broadly Programming Languages are categorized into two sections i.e. **Dynamic Programming Language** and **Strongly Typed Programming Language (Static)**. For a better understanding, please have a look at the below image.



The Strongly Typed Programming Languages are those programming languages where data type checking actually happens at compile time and Dynamically Typed Programming Languages are those languages where data type checking happens at runtime. For example, if I declared an integer variable and if I try to store some string value in it, then I will get a compile time error as shown in the below image.

```
class Program
{
    static void Main(string[] args)
    {
        int num = "Hello";
    }
}
```

Getting Compile Time Error

class System.String
Represents text as a sequence of UTF-16 code units.
Cannot implicitly convert type 'string' to 'int'

As you can see in the above image, it is saying that you **Cannot implicitly convert type 'string' to 'int'** type. So, strongly typed programming languages are those languages where type-checking happened during the compilation time.

But sometimes what happens is we don't know the object data type until the application runs. That means at the time of compilation, we don't know the data type. In other words, we confirm the object data type during the runtime only. So, in that scenario, what we need to do is, we need to bypass this compile-time type-checking logic and during runtime, we would like to go and invoke the methods and properties of the object.

But as a developer, we need to take advantage of both approaches. Because type checking is a good thing, it minimizes the defects of our application, it allows us to choose the right data type for our application.

So, yes, during compilation time, we need to bypass the type-checking. But once the type is confirmed during runtime, we need to ensure that type checking is happening. In other words, we would have something like dynamically statically typed benefits. And that is what the dynamic keyword gives us in C#. It will bypass the compile-time type checking. But once the data type is confirmed during runtime, then it will ensure that type checking is happening at runtime.

For example, if we want to declare a variable dynamically, we need to use the dynamic keyword. Here, you can see in the below image, I have created a simple object called str using the dynamic keyword. Now, you can see when we type str.(dot) it is not showing any intelligence. This thing will resolve during runtime using the concept called Reflection. So, during runtime, it will figure out exactly the data type of this str object.

```
class Program
{
    static void Main(string[] args)
    {
        dynamic str = "Hello";
        str.| No Intelligence
            It is Resolved During Runtime
    }
}
```

This is good. During compile time it will not do any kind of checking. But during runtime, once it figures out the data type, it will do the type checking. For example, you can see in the below code. On a string value, we are trying to perform a mathematical operation i.e. increment.

```
class Program
{
    static void Main(string[] args)
    {
        dynamic str = "Hello";
        str++;
    }
}
```

You can see that here we are not getting any compile time error. So, if you build the application, you will not get any errors, Build will be successful. This is because, during compile time, it is not doing any kind of checking. But during runtime, we need to ensure that this str++ operation should not work. It should throw an

exception. For a better understanding, please have a look at the below example. Here, first, we are declaring one object using the dynamic keyword. Then we are using the GetType method to get the type of the str variable and then we are performing an increment mathematical operation on the str object. The GetType method will return the type of object.

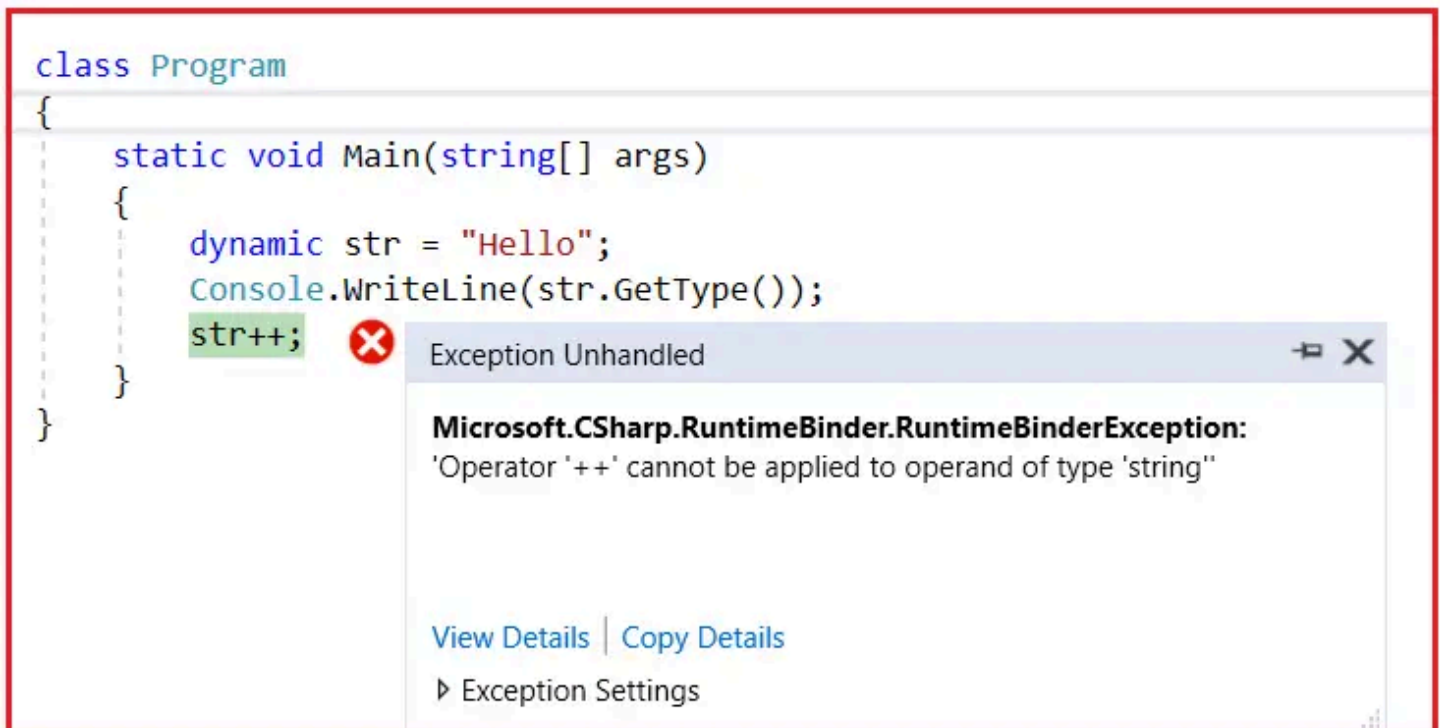
```
using System;
namespace DynamicVSReflectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            dynamic str = "Hello";
            Console.WriteLine(str.GetType());
            str++;
        }
    }
}
```

Output:

First, it will print the type of the str on the console window as follows.

`System.String`

And then immediately it will throw the following exception when the str++ statement is executed.



As you can see in the above image, it is clearly saying that **'Operator '++' cannot be applied to the operand of type 'string'**'. This is because during runtime, now it becomes strongly typed. Please

understand that during compile time, it will bypass the logic for type checking i.e. it will not check whether the increment operation is allowed or not on the str object. But runtime, it figures out the data type is a string and now if I go and invoke the mathematical increment operation on it, it should throw an exception, and that is what you can see in the above image.

So, with dynamic in C#, at compile time, we are bypassing the type-checking logic. But at runtime, we preserved the type-checking logic. Dynamic keyword uses Reflection internally. Now, I hope you understand the need and use of Dynamic Keyword in C#. Let us proceed and explore the dynamic keyword in more detail.

Why do we need Dynamic Type in C#?

In C#, we have several built-in data types such as string, int, bool, double, DateTime, etc. All these are static data types, meaning type checking and type safety are enforced only at compile time. For a better understanding, please have a look at the below example.

In the below example, first, we have declared and initialized an integer variable called "i" with the value 50. Then we created a long variable called "l" and initialize it with the value of int variable "i". The following code will compile fine and runs without any issues. This is because the int data type can be converted to a long data type without any data loss. Why because the long data type has a bigger range than the int data type. The C# compiler allows this implicit type conversion. Then we simply print the "i" and "l" value on the console.

```
using System;
namespace DynamicDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int i = 50;
            long l = i;
            Console.WriteLine($"int i = {i} & long l = {l}");
            Console.ReadKey();
        }
    }
}
```

Output: int i = 50 & long l = 50

Now, let us reverse the data type. Let us try to assign the long data type to the int data type as shown in the below example.

```
static void Main(string[] args)
{
    long l = 50;
    int i = l;
    Console.WriteLine("int i = {i} & long l = {l}");
    Console.ReadKey();
}
```

Compile Time Error

(local variable) long l

Cannot implicitly convert type 'long' to 'int'. An explicit conversion exists (are you missing a cast?)

As you can see in the above image, here we are getting a compile-time error i.e. **Cannot implicitly convert type 'long' to 'int'**. This is because an implicit conversion is not allowed in this case. The reason is long data type has a much bigger range than the int data type and there is a chance for data loss, hence the C# compiler does not allow this conversion and gives a compile-time error.

If you look at the compiler error message, the second message says that **"An explicit conversion exists (are you missing a cast?)"**. That means we can use an explicit cast if we want to convert a long data type to an int data as shown in the below example. The compiler allows this because we are doing the conversion explicitly i.e we are making the decision consciously, we know converting long data type value to int data type value can result in data loss, but in this case, the long variable has a value of 50 which can be safely converted to int data type without losing any data.

```
using System;
namespace DynamicDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            long l = 50;
            int i = (int) l; //Explicit Type Conversion
            Console.WriteLine($"int i = {i} & long l = {l}");
            Console.ReadKey();
        }
    }
}
```

Output: **int i = 50 & long l = 50**

Example to Understand Dynamic Type in C#:

Let us see another example to understand the Dynamic Type in C#. Please have a look at the following code. The following code compiles fine and runs without any errors. This is because the C# compiler knows the variable str is of type string and it has the ToUpper() instance method.

```
using System;
namespace DynamicDemo
{

```

```

class Program
{
    static void Main(string[] args)
    {
        string str = "Dynamic Keyword in C#";
        Console.WriteLine(str.ToUpper());
        Console.ReadKey();
    }
}

```

Output: DYNAMIC KEYWORD IN C#

The following code on the other hand will not compile. This is because the compiler knows that the string data type does not have an instance method called SomeMethod() and hence it will give you a compile-time error as shown in the below code.

```

using System;
namespace DynamicDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string str = "Dynamic Keyword in C#";
            str.SomeMethod(); //Compile Time Error
            Console.ReadKey();
        }
    }
}

```

So, when you try to run the above code, you will get the following compile-time error.

	Code	Description	Project	File
	CS1061	'string' does not contain a definition for 'SomeMethod' and no accessible extension method 'SomeMethod' accepting a first argument of type 'string' could be found (are you missing a using directive or an assembly reference?)	DynamicDemo	Program.cs

This compile-time checking of the code is called Static Binding or Early Binding in C# and it is a good thing because we are able to catch errors at compile-time instead of runtime. If you want to overcome this compile-time type-checking, then you need to use dynamic type in C#.

Dynamic Type in C#

This new type i.e. dynamic is introduced as part of C# 4 and as the name implies, we can use this dynamic type to write dynamic code in C#. For a better understanding, please have a look at the below example, The following code will compile fine and run without any error. Here, instead of string type, we are using dynamic type. At runtime, based on the right-hand side value, it will decide the data type of the str variable is a string and then call the ToUpper method on the string instance as the string class contains the ToUpper method.

```

using System;
namespace DynamicDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            dynamic str = "Dynamic Keyword in C#";
            Console.WriteLine(str.ToUpper());
            Console.ReadKey();
        }
    }
}

```

Output: DYNAMIC KEYWORD IN C#

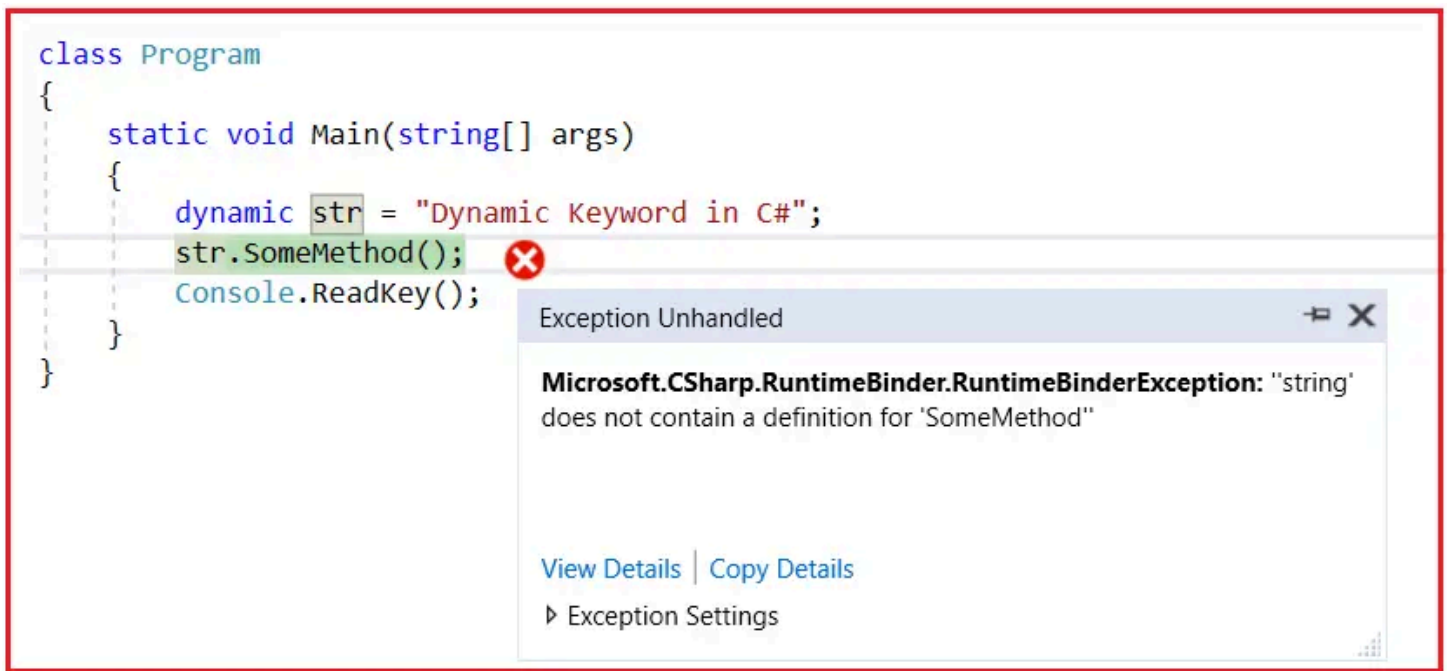
The following code will compile fine but we will get an exception at runtime. This is because, at run time, it decides the data type of the str variable is a string and the string type does not have SomeMethod(), hence it will give us a runtime exception.

```

using System;
namespace DynamicDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            dynamic str = "Dynamic Keyword in C#";
            str.SomeMethod();
            Console.ReadKey();
        }
    }
}

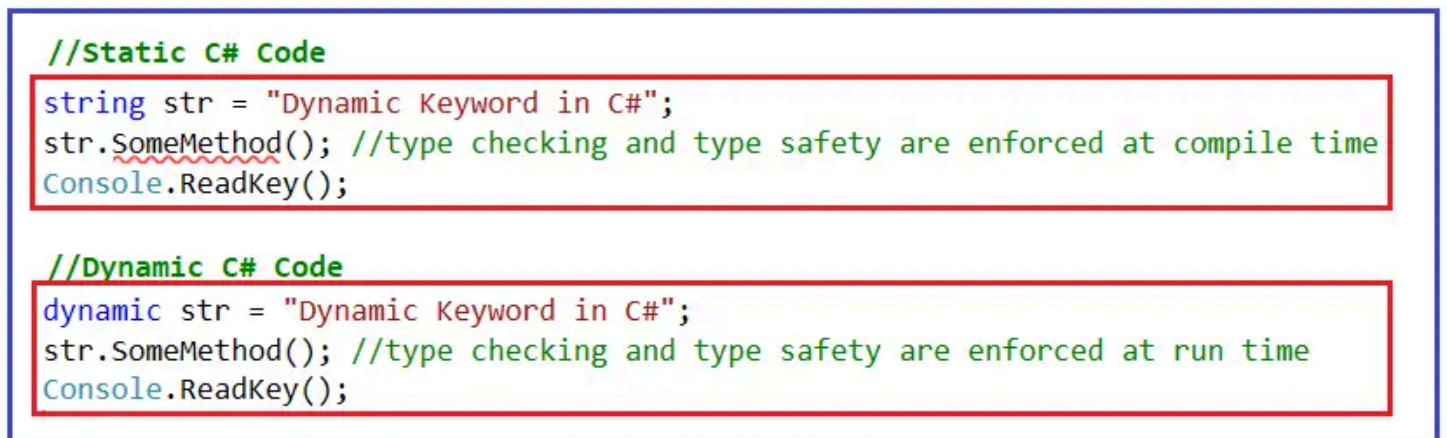
```

When you run the above code, you will get the following exception at runtime.



So, the most important point that you need to keep in mind is that even with C# dynamic data type, type checking, and type safety are also enforced. The only difference is that type checking and type safety is enforced at runtime instead of compile-time.

With static data type, the type checking and type safety are enforced at compile-time, and with the dynamic data type, type checking, and type safety are enforced at runtime. For a better understanding, please have a look at the below image. With static data type string, we are getting a compile-time error when calling the SomeMethod. But with the dynamic data type, we are not getting any compile-time error while invoking the SomeMethod.



Example to Understand Dynamic Type in C#:

So, based on the assigned value, with the dynamic data type, the CLR will decide the data type at runtime and then enforce type checking and type safety at runtime. In the below example, at runtime, it will decide the type of str as a string and the type of l as int.

```
using System;
namespace DynamicDemo
{
```

```

class Program
{
    static void Main(string[] args)
    {
        //Based on the value, at runtime it will decide the type of str as
        dynamic str = "Dynamic Keyword in C#";
        Console.WriteLine($"Type is {str.GetType()} & value = {str}");
        //Based on the value, at runtime it will decide the type of i as i
        dynamic I = 50;
        Console.WriteLine($"Type is {I.GetType()} & value = {I}");

        Console.ReadKey();
    }
}

```

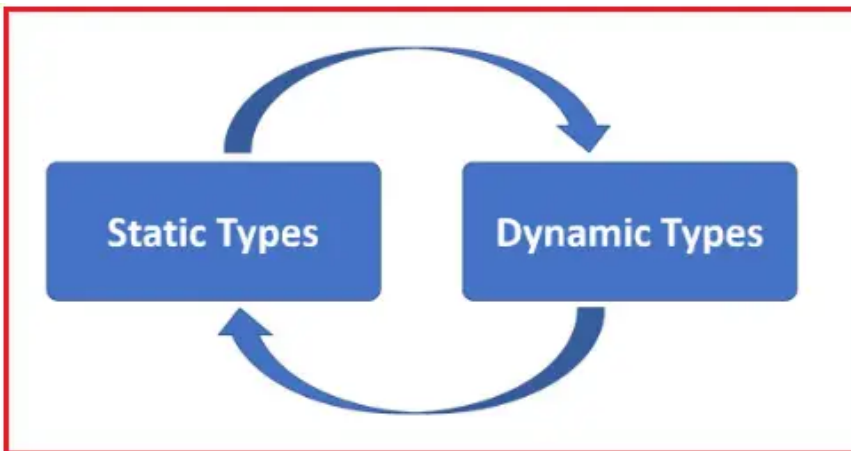
Output:

```

Type is System.String & value = Dynamic Keyword in C#
Type is System.Int32 & value = 50

```

Conversion from Static Types to Dynamic and Vice Versa in C#



In C#, conversion from static data types like int, double, float, etc, to dynamic types and vice versa does not require an explicit cast. These conversions are done implicitly. For a better understanding, please have a look at the below example. Here, we are converting the int type to dynamic type as well as dynamic type to int type without using any explicit cast operator. The following code will compile fine and run without any errors.

```

using System;
namespace DynamicDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Convert from int to dynamic

```

```

        int int1 = 50;
        dynamic dynamic1 = int1; //Explicit cast not required
        Console.WriteLine($"int1 = {int1} & dynamic1 = {dynamic1}");
        //Convert from dynamic to int
        dynamic dynamic2 = 100;
        int int2 = dynamic2; //Explicit cast not required
        Console.WriteLine($"int2 = {int2} & d2 = {dynamic2}");
        Console.ReadKey();
    }
}

```

Output:

```

int1 = 50 & dynamic1 = 50
int2 = 100 & d2 = 100

```

This is true even with complex types like Customer, Employee, etc. So, we can convert a complex type to a dynamic type as well as a dynamic type to a complex type without any error in C#.

Convert Complex Type to Dynamic Type in C#:

In the below example, we are converting the Student type to the dynamic type.

```

using System;
namespace DynamicDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Student student1 = new Student()
            {
                Name = "Anurag",
                Barnch = "CSE",
                Roll = 1001
            };
            // Student type to dynamic conversion
            dynamic dynamicStudent = student1;
            Console.WriteLine($"Name = {dynamicStudent.Name}");
            Console.WriteLine($"Barnch = {dynamicStudent.Barnch}");
            Console.WriteLine($"Roll = {dynamicStudent.Roll}");

            Console.ReadKey();
        }
    }
    public class Student
    {
        public string Name { get; set; }
    }
}

```

```

        public string Barnch { get; set; }
        public long Roll { get; set; }
    }
}

```

Output:

```

Name = Anurag
Barnch = CSE
Roll = 1001

```

Convert Dynamic Type to Complex Type in C#:

In the below example, we are converting the dynamic type to the Student type.

```

using System;
namespace DynamicDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            dynamic dynamicStudent = new Student()
            {
                Name = "Anurag",
                Barnch = "CSE",
                Roll = 1001
            };
            // dynamic to Student type conversion
            Student student1 = dynamicStudent;
            Console.WriteLine($"Name = {student1.Name}");
            Console.WriteLine($"Barnch = {student1.Barnch}");
            Console.WriteLine($"Roll = {student1.Roll}");

            Console.ReadKey();
        }
    }
    public class Student
    {
        public string Name { get; set; }
        public string Barnch { get; set; }
        public long Roll { get; set; }
    }
}

```

Output:

```
Name = Anurag  
Barnch = CSE  
Roll = 1001
```

Dynamic Type Implicit Conversions in C#:

C# allows certain data type conversions implicitly when there is no chance for data loss. For example, converting from int to double, converting from int to long, etc. Double and long have a much bigger range than int, so converting from int to long or double will not result in data loss, hence the conversion happens implicitly. This is true with both static and dynamic types in C#. For a better understanding, please have a look at the below example. The following example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;  
namespace DynamicDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            //C# Static Type Implicit Conversion  
            // int to double - implicit conversion  
            int int1 = 500;  
            double double1 = int1;  
            Console.WriteLine($"int1 = {int1} & double1 = {double1}");  
            // int to long - implicit conversion  
            int int2 = 200;  
            long long1 = int2;  
            Console.WriteLine($"int2 = {int2} & long1 = {long1}");  
            //C# Dynamic Type Implicit Conversion  
            // int to dynamic to double - implicit conversion  
            int int3 = 100;  
            dynamic dynamic1 = int3;  
            double double2 = dynamic1;  
            Console.WriteLine($"int3 = {int3} & dynamic1 = {dynamic1} & double2 = {double2}");  
            // int to dynamic to long - implicit conversion  
            int int4 = 200;  
            dynamic dynamic2 = int4;  
            long long2 = dynamic2;  
            Console.WriteLine($"int4 = {int4} & dynamic2 = {dynamic2} & long2 = {long2}");  
            Console.ReadKey();  
        }  
    }  
}
```

Output:

```
int1 = 500 & double1 = 500
int2 = 200 & long1 = 200
int3 = 100 & dynamic1 = 100 & double2 = 100
int4 = 200 & dynamic2 = 200 & long2 = 200
```

Dynamic Type Explicit Conversions in C#

In C#, converting large data types to smaller data types is not allowed implicitly by the Compiler. This is because there is a chance of data loss. In this case, we can use an explicit cast operator to do the conversion. Again, this is true for both static and dynamic data types in C#. For a better understanding, please have a look at the below example. The following code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
namespace DynamicDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            // Convert double to int. Fails to compile, an explicit cast is required
            // Error : Cannot implicitly convert type double to int
            double double1 = 4000;
            // int int1 = double1;
            // Explicit cast from double to int
            int int1 = (int)double1;
            Console.WriteLine($"double1 = {double1} & int1 = {int1}");
            // Even with dynamic an explicit cast is required when
            // converting larger data types like double to int
            double double2 = 4000;
            dynamic dynamicDouble = double2;
            int int2 = (int)dynamicDouble;
            Console.WriteLine($"double2 = {double2} & dynamicDouble = {dynamicDouble}");
            Console.WriteLine($"int2 = {int2}");
            Console.ReadKey();
        }
    }
}
```

Output:

```
double1 = 4000 & int1 = 4000
double2 = 4000 & dynamicDouble = 4000 && int2 = 4000
```

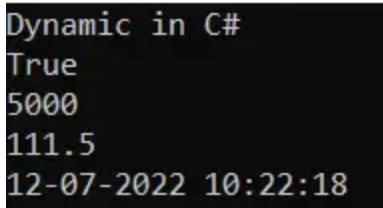
Dynamic Type as a Parameter in C#:

In C#, it is also possible to use dynamic type as a method parameter so that it can accept any type of value at run time. For a better understanding, please have a look at the following example. Here, the DisplayValue method parameter type is dynamic and hence you can see that we are passing a string, bool, double, int, etc

values to the DisplayValue method from inside the Main method. The following example will compile fine and run without any errors.

```
using System;
namespace DynamicDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Calling DisplayValue Function with different types of values
            DisplayValue("Dynamic in C#"); //String
            DisplayValue(true); //Boolean
            DisplayValue(5000); //Integer
            DisplayValue(111.50); //Double
            DisplayValue(DateTime.Now); //Date
            Console.ReadKey();
        }
        public static void DisplayValue(dynamic val)
        {
            Console.WriteLine(val);
        }
    }
}
```

Output:



```
Dynamic in C#
True
5000
111.5
12-07-2022 10:22:18
```

Why do we need Dynamic Type in C#?

By looking at the examples we have discussed so far, you might be thinking, why do we need dynamic type in C#, and what benefits it provides? The dynamic type C# has provided several benefits. They are as follows:

1. **Simplifies Processing of JSON API Data:** In general, when an API returns JSON data, we normally create another strong type class in our application and map the JSON data to that strongly typed class. However, in some scenarios where we do not want to create yet another strongly type class but still want to be able to consume and process the JSON data, we can make use of dynamic type in C#. In our upcoming article, we will see this with a real-time example.
2. **Interoperate with other languages like IronRuby or IronPython:** Dynamic in C# makes it possible to interoperate with other programming languages like IronRuby or IronPython. If you are wondering, why do we need to interoperate with other programming languages? Well, to use features of other languages that C# doesn't support.

Dynamic Type Real-time Example in C#:

With dynamic type in C#, it is very easy to write reflection code which in turn makes the code more readable and maintainable. Let us see an example for a better understanding. We want to invoke an instance method using reflection in C#. Please read our previous article where we discussed [Reflection](#) in Detail. Here, I am not going to explain anything related to Reflection, rather I am simply going to use Reflection. Please have a look at the following Calculator class.

```
public class Calculator
{
    public int Add(int number1, int number2)
    {
        return number1 + number2;
    }
}
```

The above class is very straightforward. This class had one method i.e. Add which takes two integer parameters and the Add method returns the sum of the two input numbers. Now, we want to invoke the above Add method using Reflection. To invoke the above Add method using Reflection we need to write the following code.

```
Calculator calculator = new Calculator();

//Using Reflection to Invoke the Add method
var result = calculator.GetType().InvokeMember("Add",
    System.Reflection.BindingFlags.InvokeMethod,
    null,
    calculator,
    new object[] { 10, 20 });

Console.WriteLine($"Sum = {result}");
```

The complete example code is given below.

```
using System;
namespace DynamicDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Calculator calculator = new Calculator();
            //Using Reflection to Invoke the Add method
            var result = calculator.GetType().InvokeMember("Add",
                System.Reflection.BindingFlags.InvokeMethod,
```

```

        null,
        calculator,
        new object[] { 10, 20 });
    Console.WriteLine($"Sum = {result}");
    Console.ReadKey();
}
}
public class Calculator
{
    public int Add(int number1, int number2)
    {
        return number1 + number2;
    }
}
}

```

Output: **Sum = 30**

Here, as you can see, we have written a lot of code just to call the Add method using Reflection. The code size not only big but also it is complex and difficult to understand. The above reflection code can be rewritten using dynamic. And using dynamic, the code will be simpler, cleaner, and easier to understand. The following example uses dynamic to invoke the Add method.

```

using System;
namespace DynamicDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            dynamic calculator = new Calculator();
            var result = calculator.Add(10, 20);
            Console.WriteLine($"Sum = {result}");
            Console.ReadKey();
        }
    }
    public class Calculator
    {
        public int Add(int number1, int number2)
        {
            return number1 + number2;
        }
    }
}

```

Output: **Sum = 30**

Limitations of Dynamic Type in C#:

In most situations, it is not advisable to use the dynamic type unless you are integrating with a dynamic language or another framework where types are not known at compile time. Because the compiler does not know what type the dynamic variable will eventually become, it's unable to offer method or property code hints in Visual Studio.

In the next article, I am going to discuss [Var Keyword in C#](#) with Examples. Here, in this article, I try to explain **Dynamic Type in C#** with Examples. I hope you enjoy this Dynamic Type in C# with Examples article. I would like to have your feedback. Please post your feedback, question, or comments about this article.